



南開大學
Nankai University

建模与辨识课程 作业报告

题 目： 基于麦夸特算法的
 飞行器动力学模型参数辨识
姓 名： 殷家兴
学 号： 2313666
专 业： 自动化

人工智能学院

2026 年 6 月

目录

一 数据来源	1
二 麦夸特算法原理与推导	2
2.1 麦夸特算法基本原理	2
2.2 算法推导	2
三 算法实现	3
3.1 辨识参数的选取	3
3.2 关键代码解释	3
四 实验结果与分析	6
附录	8
A Marquardt.py	8

一 数据来源

本次作业的数据来源延续第一次作业《基于卡尔曼滤波器的飞行器轨迹跟踪状态估计》中所使用的多涵道飞行器轨迹跟踪仿真数据。

前期工作已经得到了一个训练好的 Policy 网络，本文基于该网络的正弦轨迹跟踪控制仿真实验数据，使用麦夸特算法实现对非线性系统的参数辨识，并将得到的状态估计与仿真中的实际状态数据进行对比。

飞行器的动力学方程为

$$\begin{cases} \dot{v} = \frac{F_l + F_r - F_{\text{drag}}}{m}, \\ \dot{\omega} = \frac{(F_r - F_l)l - 2M_{\text{drag}}}{2I_{zz}}, \\ \dot{\varphi} = \omega, \\ \dot{x} = v \cos \varphi, \\ \dot{y} = v \sin \varphi, \end{cases} \quad (1)$$

其离散化形式为

$$\mathbf{x}(k+1) = \begin{bmatrix} v(k) \sin \varphi(k) \\ \omega(k) \\ \frac{(F_r(k) - F_l(k))l - 2M_{\text{drag}}(k)}{2I_{zz}} \\ \frac{F_l(k) + F_r(k) - F_{\text{drag}}(k)}{m} \\ v(k) \cos \varphi(k) \end{bmatrix} T_s + \mathbf{x}(k) \quad (2)$$

其中， $F_{\text{drag}} = 0.5\rho C_d A(\varphi)v|v| + c_v v$ 、 $M_{\text{drag}} = c_\omega \omega$ 。

下面再次给出在第一次作业报告中给出的飞行器动力学模型（图1）与系统参数表（表1）。

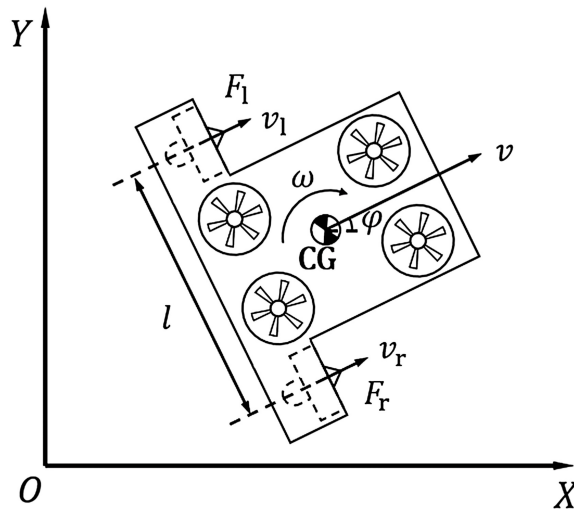


图 1: 多涵道飞行器动力学模型

表 1: 原系统参数表

Parameter	Value	Parameter	Value
m	2.7kg	ρ	1.2kg/m ³
I_{zz}	0.07kg · m ²	C_d	1.2
l	0.3m	c_ω	0.08kg · m ² /s
L_1	0.36m	c_v	0.5kg/s
L_b	0.25m	T_s	0.05s
h	0.05m		

二 麦夸特算法原理与推导

2.1 麦夸特算法基本原理

麦夸特方法是用于解决非线性最小二乘问题的一种迭代算法。

考虑如下目标函数：

$$J(\theta) = \sum_{k=1}^N [y(k) - f(u(k), \theta)]^2 \quad (3)$$

其中 $y(k)$ 是观测值， $f(u(k), \theta)$ 是关于参数 θ 的非线性函数。

目标是通过迭代优化参数 θ ，使 $J(\theta)$ 最小。

麦夸特方法结合了高斯-牛顿法（快速但可能发散）和梯度下降法（稳定但收敛慢）的优点，通过引入一个可调节的参数 λ ，在保证收敛的前提下尽可能加快收敛速度。

2.2 算法推导

在 $\theta = \theta^{(i)}$ 处将 $y = f(u(k), \theta)$ 一阶泰勒展开

$$y = f(u, \theta) \approx f(u, \theta^{(i)}) + \left(\frac{\partial f}{\partial \theta}\right)^\top|_{\theta=\theta^{(i)}}(\theta - \theta^{(i)}) \quad (4)$$

将式 (4) 代入式 (3)，得到

$$J(\theta) = \sum_{k=1}^N [y(k) - f(u, \theta^{(i)}) - \left(\frac{\partial f}{\partial \theta}\right)^\top|_{\theta=\theta^{(i)}}(\theta - \theta^{(i)})]^2 \quad (5)$$

对 θ 求偏导

$$\frac{\partial J}{\partial \theta} = -2 \sum_{k=1}^N \left(\frac{\partial f}{\partial \theta}\right)|_{\theta=\theta^{(i)}} [y(k) - f(u(k), \theta^{(i)}) - \left(\frac{\partial f}{\partial \theta}\right)^\top|_{\theta=\theta^{(i)}}(\theta - \theta^{(i)})] = 0 \quad (6)$$

令 $\frac{\partial J}{\partial \theta} = 0$ ，得到

$$\sum_{k=1}^N \left(\frac{\partial f}{\partial \theta}\right)|_{\theta=\theta^{(i)}} \left(\frac{\partial f}{\partial \theta}\right)^\top|_{\theta=\theta^{(i)}}(\theta - \theta^{(i)}) = \sum_{k=1}^N \left(\frac{\partial f}{\partial \theta}\right)|_{\theta=\theta^{(i)}} [y(k) - f(u(k), \theta^{(i)})] \quad (7)$$

令 $\sum_{k=1}^N (\frac{\partial f}{\partial \theta})|_{\theta=\theta^{(i)}} (\frac{\partial f}{\partial \theta})^\top|_{\theta=\theta^{(i)}} (\theta - \theta^{(i)}) = A_i$, 易知 A_i 半正定, 则

$$\theta^{(i+1)} = \theta^{(i)} + A_i^{-1} \sum_{k=1}^N (\frac{\partial f}{\partial \theta})|_{\theta=\theta^{(i)}} [y(k) - f(u(k), \theta^{(i)})] \quad (8)$$

令 $G_i = \frac{\partial J}{\partial \theta}|_{\theta=\theta^{(i)}} = -2 \sum_{k=1}^N (\frac{\partial f}{\partial \theta})|_{\theta=\theta^{(i)}} [y(k) - f(u(k), \theta^{(i)})]$, 则

$$\theta^{(i+1)} = \theta^{(i)} - \frac{1}{2} A_i^{-1} G_i \quad (9)$$

为保证 A_i 可逆且方向可接受, 引入 λ 对 $\theta^{(i+1)}$ 进行改造

$$\theta^{(i+1)} = \theta^{(i)} - \frac{1}{2} (A_i + \lambda I)^{-1} G_i \quad (10)$$

其中: λ 充分大时一定可接受, 但是与原式差异较大; λ 充分小时, 若可接受则与原式相差不大且收敛快, 若不可接受则无法求解。

三 算法实现

3.1 辨识参数的选取

根据飞行器动力学模型, 选取以下五个参数作为辨识对象。

表 2: 辨识参数选取

参数符号	物理意义	真值	辨识范围
m	飞行器质量	2.7kg	[1.5, 4.0]
I_{zz}	横摆转动惯量	0.07kg · m ²	[0.04, 0.12]
C_d	空气阻力系数	1.2	[0.5, 1.8]
c_ω	角速度阻尼系数	0.08kg · m ² /s	[0.03, 0.15]
c_v	粘性阻尼系数	0.5kg/s	[0.2, 0.8]

以上参数涵盖了飞行器的惯性特性和气动阻尼特性, 是影响跟踪精度的关键因素。

因此有

$$\theta = [m \ I_{zz} \ C_d \ c_\omega \ c_v] \quad (11)$$

设置初始参数猜测值

$$\theta^{(0)} = [m_0 \ I_{zz0} \ C_{d0} \ c_{\omega0} \ c_{v0}] = [1.50 \ 0.05 \ 1.00 \ 0.05 \ 0.10] \quad (12)$$

3.2 关键代码解释

当前系统已经有训练好的 Policy 网络作为系统模型, 通过使用模型获取飞行器跟踪正弦轨迹的输入输出仿真数据。基于所得数据, 对系统参数进行辨识。

动力学仿真

```
def _state_function_numpy(self, state, action, params_dict):
    """状态导数计算"""
    mass = params_dict['mass']
    Izz = params_dict['Izz']
    Cd = params_dict['Cd_horizontal']
    # 空气阻力计算
    effective_area = thickness * (Ll*abs(cos(psi)) + Lb*abs(sin(psi))
    ))
    F_drag = 0.5 * rho * Cd * effective_area * abs(v) * v + k_vis *
    v
    # 加速度计算
    deriv_u = (F_left + F_right - F_drag) / mass
    deriv_omega = (d*(F_right-F_left) - k_ang*omega) / Izz
    return np.array([deriv_y, deriv_psi, deriv_omega, deriv_u,
    deriv_x])
```

该函数实现了连续时间状态导数的计算，与原系统动力学模块中的状态更新函数保持数学形式一致，确保了仿真模型的准确性。

雅可比矩阵计算

```
def compute_jacobian(self, params_array, ...):
    for i in range(n_params):
        params_plus = params_array.copy()
        params_plus[i] += eps
        r_plus = self.compute_residuals(params_plus, ...)
        params_minus = params_array.copy()
        params_minus[i] -= eps
        r_minus = self.compute_residuals(params_minus, ...)
        J[:, i] = (r_plus - r_minus) / (2 * eps)
    return J, r0
```

采用中心差分法计算雅可比矩阵，步长 $\epsilon = 10^{-6}$ ，在计算精度和效率之间取得平衡。

收敛判断

```
delta_norm = np.linalg.norm(delta)
if delta_norm < tolerance: # tolerance = 1e-4
```

```
print(f"收敛条件满足:  $\|\Delta\| = \{\text{delta\_norm:.4e}\}")$ 
break
```

当参数更新量的范数小于容差时判定收敛，防止无效迭代。

参数更新与阻尼因子自适应

```
def levenberg_marquardt(self, u_sequence, measured_outputs,
    initial_state,
                                params0=None, lambda_init=0.1, lambda_factor
                                =4.0,
                                max_iter=50, tolerance=1e-7, verbose=True):

    # 1. 初始化参数和阻尼因子
    theta = params0.copy()      #
    lam = lambda_init           # = 0.1

    # 2. 计算初始残差和目标函数
    r = self.compute_residuals(theta, u_sequence, measured_outputs,
        initial_state)
    J = 0.5 * np.sum(r**2)      #  $J = \frac{1}{2} \|r\|^2$ 

    for iteration in range(max_iter):
        # 3. 计算雅可比矩阵和梯度
        J_acobian, r = self.compute_jacobian(theta, ...)
        JTJ = J_acobian.T @ J_acobian    #  $A = J^T J$ 
        JTr = J_acobian.T @ r            #  $g = J^T r$ 

        # 4. 尝试步长（内循环：调整直到接受更新）
        accepted = False
        while not accepted and inner_iter < max_inner:
            # 核心公式：求解  $(J^T J + I)\Delta = J^T r$ 
            A_reg = JTJ + lam * np.eye(len(theta))
            delta = np.linalg.solve(A_reg, JTr)    #  $\Delta = (J^T J + I)^{-1} J^T r$ 

            # 试探新参数
            theta_new = theta - delta              #  $\theta_{\text{new}} = \theta - \Delta$ 
            theta_new = self.clamp_params(theta_new) # 边界约束
```

```

# 计算新目标函数
r_new = self.compute_residuals(theta_new, ...)
J_new = 0.5 * np.sum(r_new**2)

# 5. 判断是否接受更新
if J_new < J:
    # 接受：更新参数，减小
    theta = theta_new
    J = J_new
    lam = max(lam / lambda_factor, 1e-8) # ← /4
    accepted = True
else:
    # 拒绝：增大，重新计算
    lam *= lambda_factor # ← 4
    inner_iter += 1

# 6. 收敛判断
delta_norm = np.linalg.norm(delta)
if delta_norm < tolerance:
    break

```

参数更新的核心是求解一个线性方程组得到参数调整量，然后按当前参数减去该调整量的方式获得新参数。

阻尼因子根据目标函数的变化进行自适应调节：如果目标函数下降，说明当前步长方向正确，此时减小阻尼因子使算法趋向快速的高斯-牛顿法以加速收敛；如果目标函数上升，说明步长过大或方向不佳，此时增大阻尼因子使算法趋向稳定的梯度下降法，并重新计算步长。这种自适应调节机制在保证算法稳定收敛的前提下尽可能采用快速收敛方向，是麦夸特算法的精髓所在。

完整的麦夸特算法实现代码见附录。

四 实验结果与分析

使用 500 步仿真数据进行参数辨识，算法在第 92 次迭代满足收敛条件，得到最终目标函数 $J = 1.165703e - 01$ 、最终阻尼因子 $\lambda = 0.025$ 。

最终的参数辨识结果如表3和图2所示。

可以看到，质量 m 、转动惯量 I_{zz} 、角速度阻尼系数 c_ω 和粘性阻尼系数 c_v 的辨识误差均在 5% 以内，后三个参数的误差甚至低于 1%，这表明麦夸特算法对这些参数的敏感度较高，能够通过轨迹跟踪数据准确反演。

而空气阻力系数 C_d 的辨识结果与真值偏差达 50%，这可能是因为飞行器在正弦

轨迹跟踪过程中速度变化范围较小（约 0.3-0.45 m/s），空气阻力与速度平方成正比，低速条件下阻力占比较小，导致该参数的可辨识性不足。此外，质量与阻力系数之间存在耦合效应——两者同时影响纵向加速度，仅凭位置测量难以完全解耦。

表 3: 参数辨识结果

参数符号	真值	辨识值
m	2.7000	2.7762
I_{zz}	0.0700	0.0678
C_d	1.2000	1.8000
c_ω	0.0800	0.0794
c_v	0.5000	0.4990

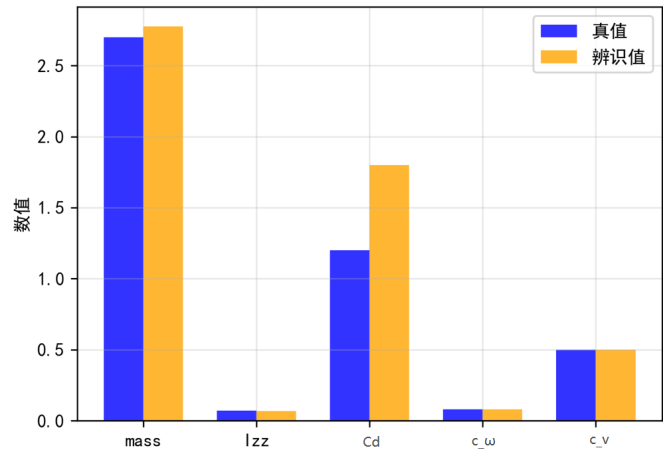


图 2: 参数辨识结果

除 C_d 外，其余四个参数均获得满意结果，验证了麦夸特算法在飞行器参数辨识中的有效性。

图3和图4展示了辨识模型的横向位置 y 与航向角 ϕ 随纵向位置 x 的变化规律与真实轨迹的对比情况。从图中也可以看出，通过麦夸特算法辨识出的参数结果较为准确。

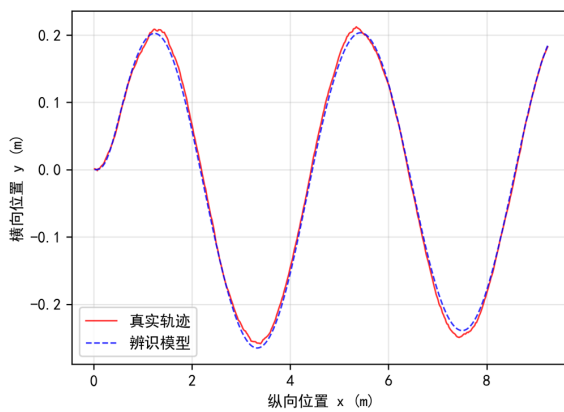


图 3: 横向位置辨识模型与真实轨迹对比

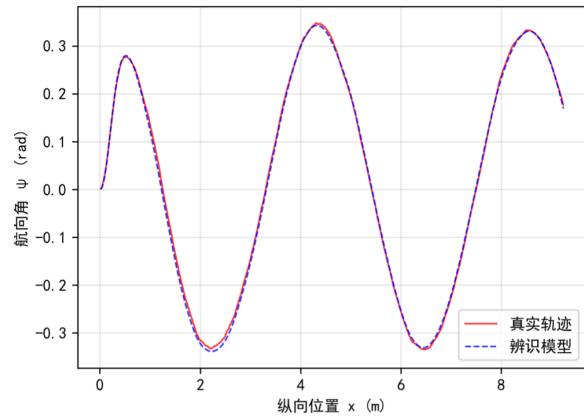


图 4: 航向角辨识模型与真实轨迹对比

附录

A Marquardt.py

```
import numpy as np
import torch

class LMAIdentifier:
    """麦夸特算法参数辨识器"""
    def __init__(self, dynamics, dt=0.05):
        self.dynamics = dynamics
        self.dt = dt
        self.device = dynamics.device
        self.rho = dynamics.rho
        self.thickness = dynamics.thickness
        self.Ll = dynamics.Ll
        self.Lb = dynamics.Lb
        self.d = dynamics.d
        # 参数名称和边界
        self.param_names = ['mass', 'Izz', 'Cd_horizontal', 'k_angular_drag', 'k_viscous_base']
        self.param_bounds = {
            'mass': (1.5, 4.0),
            'Izz': (0.04, 0.12),
            'Cd_horizontal': (0.5, 1.8),
            'k_angular_drag': (0.03, 0.15),
            'k_viscous_base': (0.2, 0.8)
        }
        # 存储真值（用于对比）
        self.true_params = {
            'mass': dynamics.mass,
            'Izz': dynamics.Izz,
            'Cd_horizontal': dynamics.Cd_horizontal,
            'k_angular_drag': dynamics.k_angular_drag,
            'k_viscous_base': dynamics.k_viscous_base
        }
        # 迭代历史
        self.history = {
            'theta': [],
```

```

        'lambda': [],
        'J': [],
        'grad_norm': []
    }

def get_params_array(self, params_dict):
    """参数字典转数组"""
    return np.array([params_dict[name] for name in self.
        param_names])

def get_params_dict(self, params_array):
    """参数数组转字典"""
    return {name: params_array[i] for i, name in enumerate(self.
        param_names)}

def clamp_params(self, params_array):
    """参数边界约束"""
    for i, name in enumerate(self.param_names):
        low, high = self.param_bounds[name]
        params_array[i] = np.clip(params_array[i], low, high)
    return params_array

def _state_function_numpy(self, state, action, params_dict):
    """状态导数计算"""
    y, psi, omega, v, x = state
    F_left, F_right = action
    # 限制推力
    F_left = np.clip(F_left, -50, 50)
    F_right = np.clip(F_right, -50, 50)
    # 解包参数
    mass = params_dict['mass']
    Izz = params_dict['Izz']
    Cd = params_dict['Cd_horizontal']
    k_ang = params_dict['k_angular_drag']
    k_vis = params_dict['k_viscous_base']
    # 力和力矩
    F_total = F_left + F_right
    delta_F = F_right - F_left
    # 空气阻力

```

```

        effective_area = self.thickness * (self.Ll * abs(np.cos(psi))
            + self.Lb * abs(np.sin(psi)))
        F_drag = 0.5 * self.rho * Cd * effective_area * abs(v) * v +
            k_vis * v
        # 加速度
        deriv_u = (F_total - F_drag) / mass
        M_z = self.d * delta_F
        angular_damping = k_ang * omega
        deriv_omega = (M_z - angular_damping) / Izz
        # 运动学
        deriv_y = v * np.sin(psi)
        deriv_psi = omega
        deriv_x = v * np.cos(psi)
        return np.array([deriv_y, deriv_psi, deriv_omega, deriv_u,
            deriv_x])

def simulate_trajectory(self, u_sequence, initial_state,
    params_dict):
    """仿真轨迹"""
    N = len(u_sequence)
    states = np.zeros((N, 5))
    state = initial_state.copy()
    for i in range(N):
        # 获取当前控制
        action = u_sequence[i]
        # 计算导数
        deriv = self._state_function_numpy(state, action,
            params_dict)
        # 欧拉积分
        state = state + self.dt * deriv
        states[i] = state.copy()
    return states

def compute_residuals(self, params_array, u_sequence,
    measured_outputs, initial_state):
    """计算残差"""
    params_dict = self.get_params_dict(params_array)
    states = self.simulate_trajectory(u_sequence, initial_state,
        params_dict)

```

```

model_y = states[:, 0]
model_psi = states[:, 1]
residuals_y = model_y - measured_outputs[:, 0]
residuals_psi = model_psi - measured_outputs[:, 1]
return np.concatenate([residuals_y, residuals_psi])

def compute_jacobian(self, params_array, u_sequence,
measured_outputs, initial_state, eps=1e-6):
    """计算雅可比矩阵"""
    n_params = len(params_array)
    r0 = self.compute_residuals(params_array, u_sequence,
        measured_outputs, initial_state)
    J = np.zeros((len(r0), n_params))
    for i in range(n_params):
        params_plus = params_array.copy()
        params_plus[i] += eps
        r_plus = self.compute_residuals(params_plus, u_sequence,
            measured_outputs, initial_state)
        params_minus = params_array.copy()
        params_minus[i] -= eps
        r_minus = self.compute_residuals(params_minus,
            u_sequence, measured_outputs, initial_state)
        J[:, i] = (r_plus - r_minus) / (2 * eps)
        print(f"    参数 {self.param_names[i]}: 完成")
    return J, r0

def levenberg_marquardt(self, u_sequence, measured_outputs,
initial_state,
                        params0=None, lambda_init=0.1,
                        lambda_factor=2.0,
                        max_iter=50, tolerance=1e-7, verbose=
                        True):
    """麦夸特算法主循环"""
    # 初始化
    if params0 is None:
        params0 = np.array([2.5, 0.065, 1.1, 0.07, 0.4])
    else:
        params0 = self.clamp_params(params0)
    theta = params0.copy()

```

```

lam = lambda_init
best_theta = theta.copy()
best_J = float('inf')
# 重置历史
self.history = {'theta': [theta.copy()], 'lambda': [lam], 'J': [], 'grad_norm': []}
# 计算初始J
r = self.compute_residuals(theta, u_sequence,
                           measured_outputs, initial_state)
J = 0.5 * np.sum(r**2)
self.history['J'].append(J)
for iteration in range(max_iter):
    # 计算雅可比和梯度
    J_acobian, r = self.compute_jacobian(theta, u_sequence,
                                          measured_outputs, initial_state)
    JTJ = J_acobian.T @ J_acobian
    JTr = J_acobian.T @ r
    grad_norm = np.linalg.norm(JTr)
    self.history['grad_norm'].append(grad_norm)
    # 尝试步长
    accepted = False
    inner_iter = 0
    max_inner = 15
    while not accepted and inner_iter < max_inner:
        A_reg = JTJ + lam * np.eye(len(theta))
        try:
            delta = np.linalg.solve(A_reg, JTr)
        except np.linalg.LinAlgError:
            lam *= lambda_factor
            inner_iter += 1
            continue
        theta_new = theta - delta
        theta_new = self.clamp_params(theta_new)
        # 计算新J
        r_new = self.compute_residuals(theta_new, u_sequence,
                                       measured_outputs, initial_state)
        J_new = 0.5 * np.sum(r_new**2)
        # 判断是否接受
        if J_new < J:

```

```

        theta = theta_new
        J = J_new
        lam = max(lam / lambda_factor, 1e-8)
        accepted = True
        if J < best_J:
            best_theta = theta.copy()
            best_J = J
        else:
            lam *= lambda_factor
            inner_iter += 1

# 记录历史
self.history['theta'].append(theta.copy())
self.history['lambda'].append(lam)
self.history['J'].append(J)

# 收敛判断
delta_norm = np.linalg.norm(delta)
if delta_norm < tolerance:
    if verbose:
        print(f"\n收敛条件满足: ||Δ|| = {delta_norm:.4e} < {tolerance}")
    break
return best_theta, self.history

def get_history(self):
    """获取迭代历史"""
    return self.history

def predict_with_params(self, params_dict, u_sequence,
                        initial_state):
    """使用给定参数仿真轨迹"""
    return self.simulate_trajectory(u_sequence, initial_state,
                                    params_dict)

```